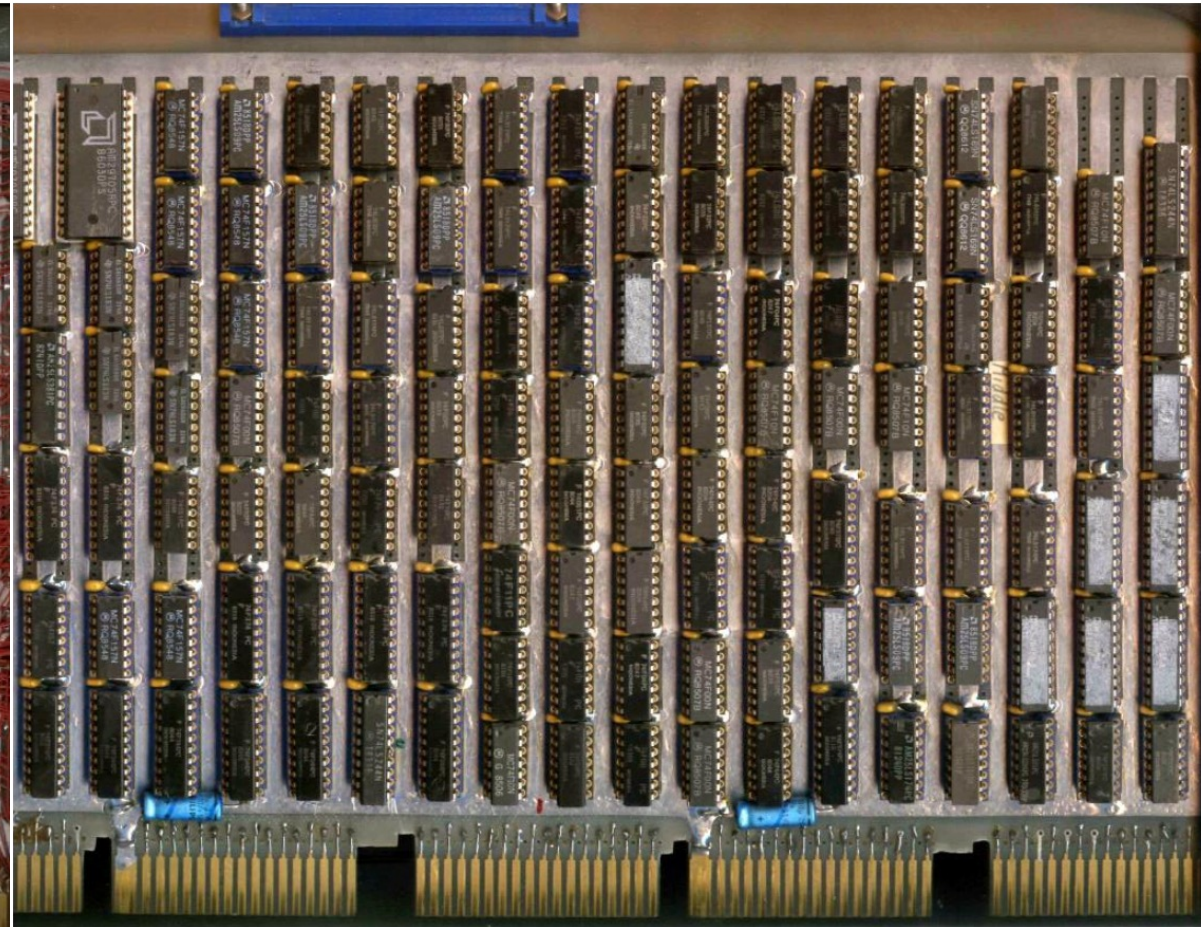
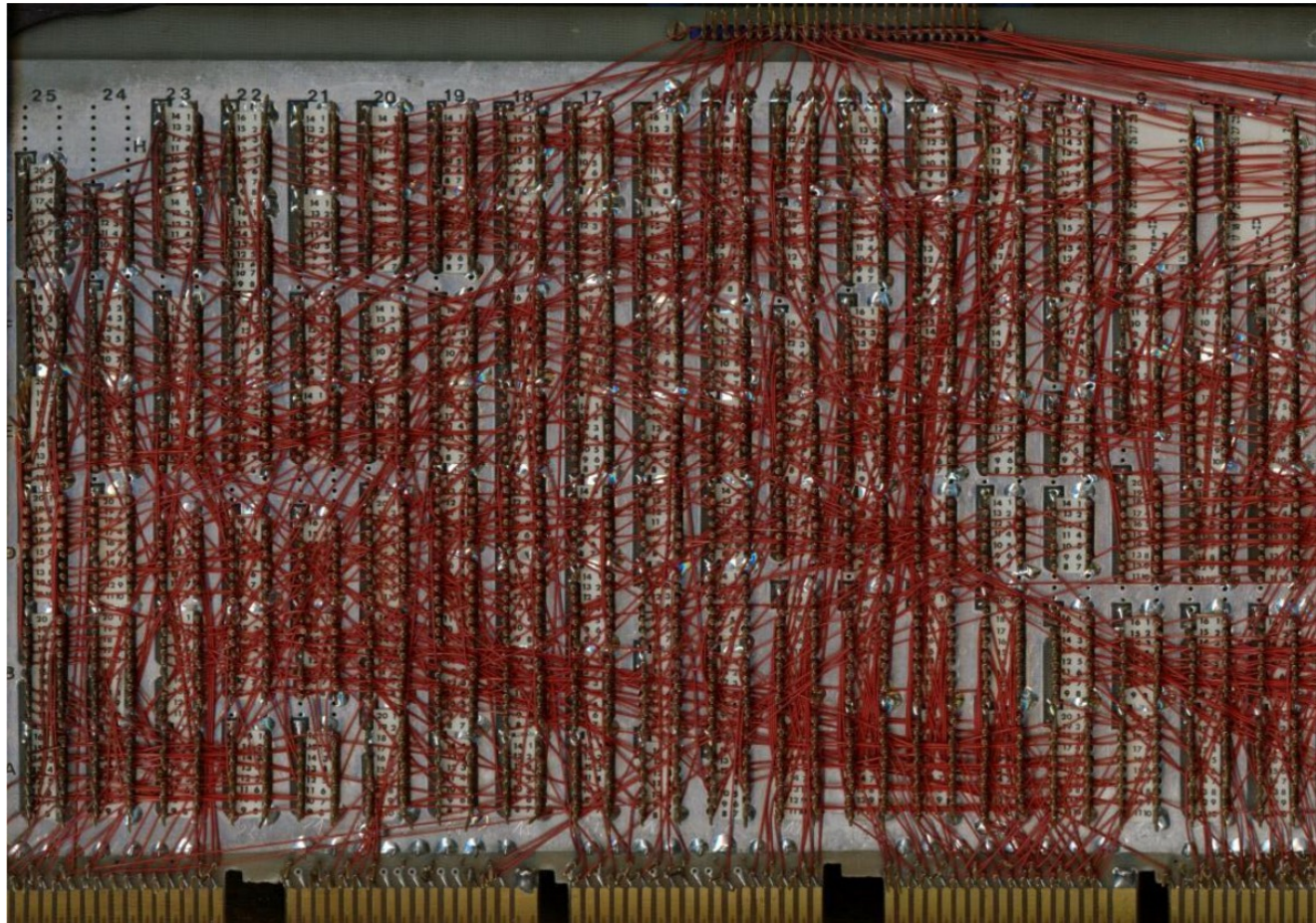


# **Lezione 4**

## ***Introduction to FPGAs and Xilinx devices***

Stefan Cristi Zugravel  
20/01/2026

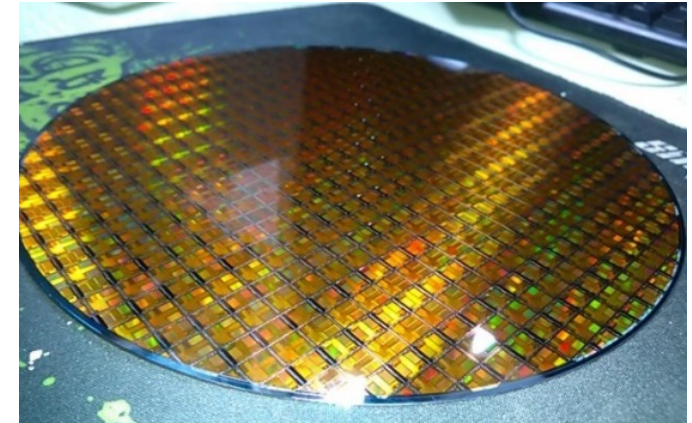
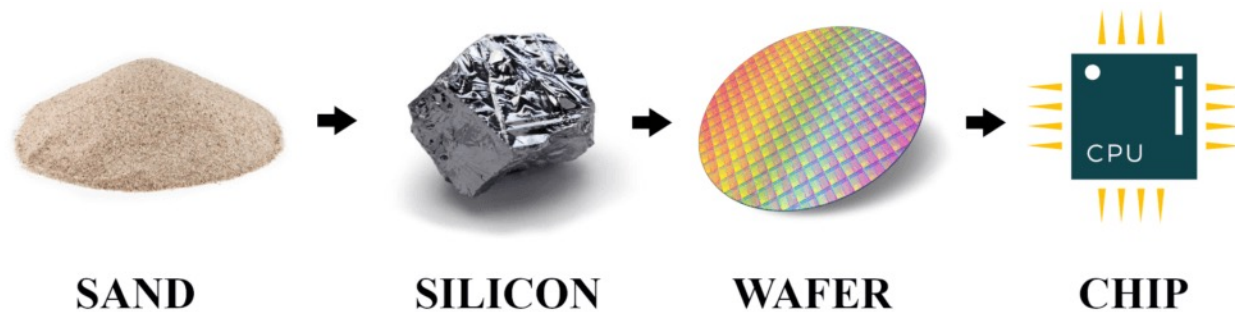
# Early times of digital electronics





# Why programmable ICs?

- Producing a Chip costs much more than producing a PCB
- Using one Chip on many PCBs increases Number of Chips and reduces Costs per Chip
- If a Chip is programmable, it can be used on many PCBs in different situations
- Producing one wafer costs ~ 10k€
  - Not included manpower to design wafer & packaging
- Saving space on a PCB



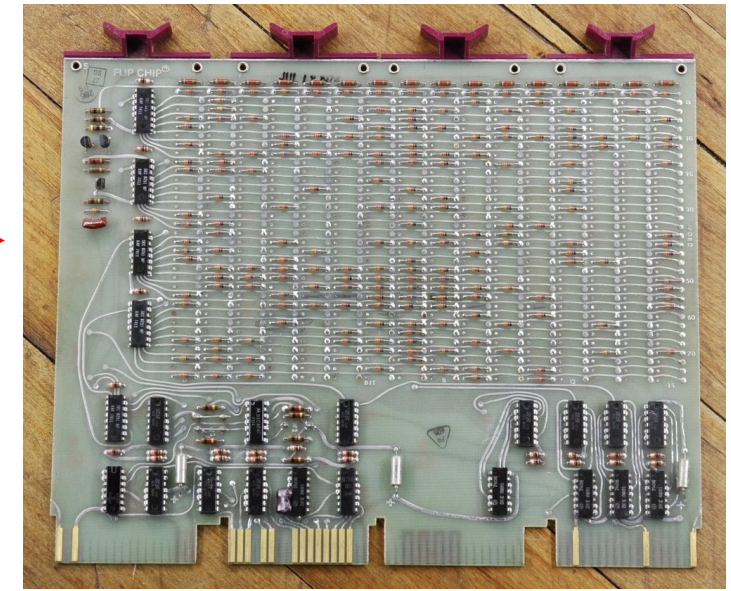
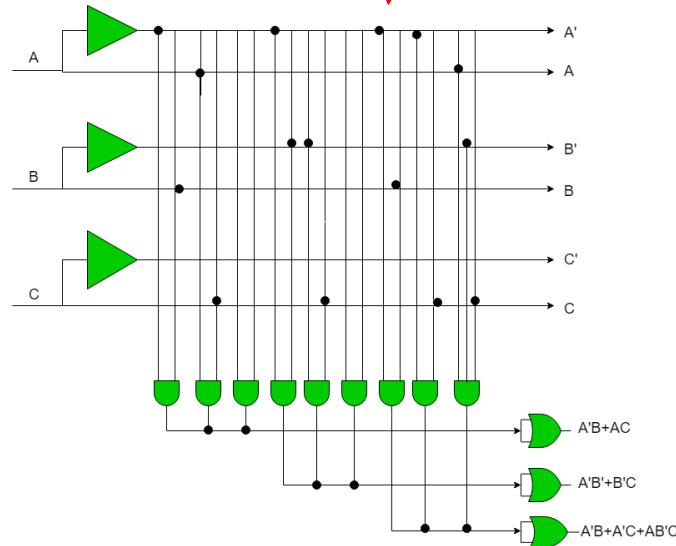
# What are FPGAs?

## Field Programmable Gate Array

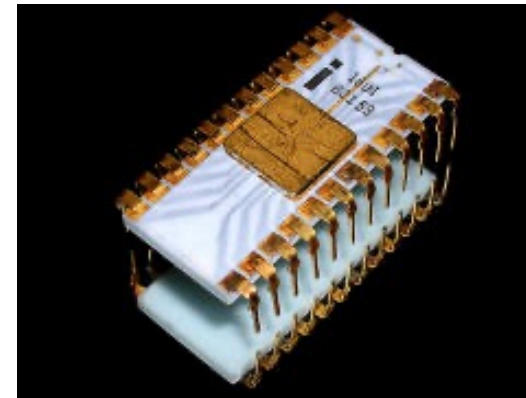
- Simple:
  - Programmable Logic Devices
- More detailed:
  - Programmable Logic Devices
    - With lot of additional function-block (e.g. RAM, DSP)
    - Flexible configuration
    - Can host microcontroller

# History of Programmable Logic Devices /1

- **1960s:** Fuse Configurable Diode Matrix
  - First form of programmable logic
  - One-Time Programmable (OTP)
- **1971:** Programmable ROM (**PROM**)
  - Logic functions stored as truth tables
  - Still limited flexibility and efficiency
- **PAL** (Programmable Array Logic):
  - Fixed AND/OR logic structure
  - Programmable interconnections
  - OTP technology

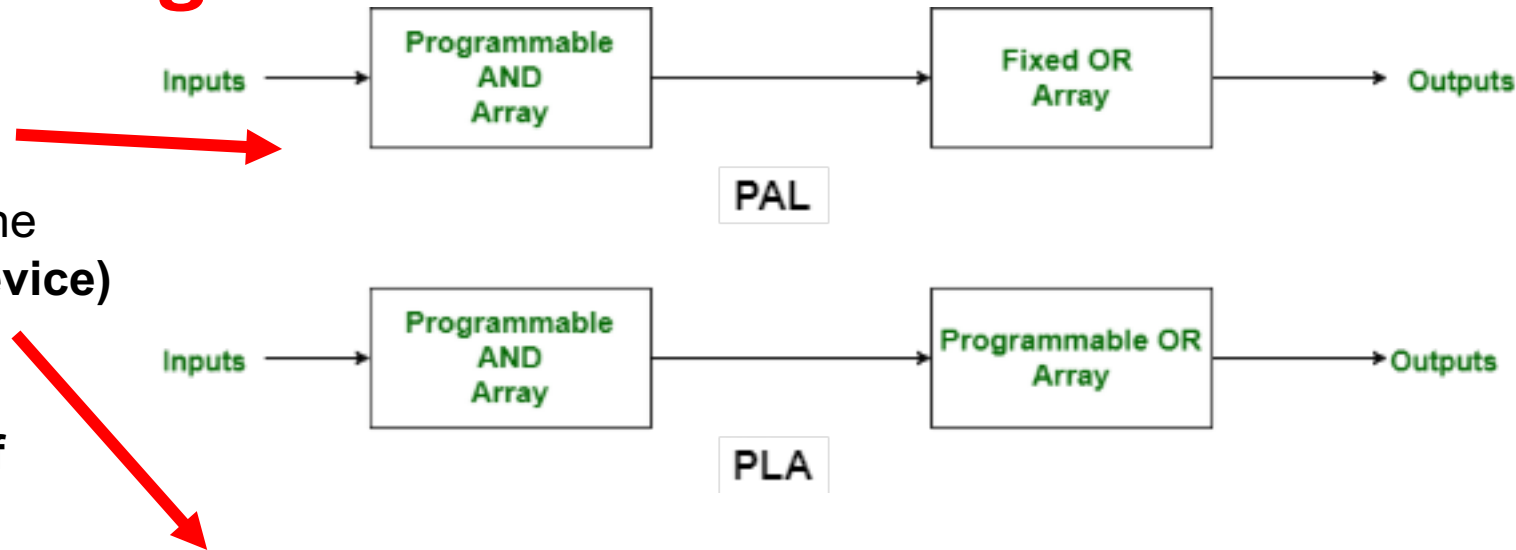


## 256 x 8-bit Static/Dynamic PROM



# History of Programmable Logic Devices /2

- **1978: PLA (Programmable Logic Array)**
  - Programmable AND plane and OR plane
- **CPLD (Complex Programmable Logic Device)**
  - Multiple PAL-like blocks
  - In-system programmable
  - Configuration **retained after power-off**



The most noticeable difference between a large CPLD and a small FPGA is the presence of on-chip non-volatile memory in the CPLD, which allows CPLDs to be used for bootloader functions

## Comparison with other Programmable Logic Devices

- PLA has a programmable AND gate array and programmable OR gate array.
- PAL has a programmable AND gate array but a fixed OR gate array.
- ROM has a fixed AND gate array but programmable OR gate array.



# History of Programmable Logic Devices /3

- **1989:** First Field Programmable Gate Array (FPGA)
  - Configurable Logic Blocks (CLBs)
  - Programmable routing fabric
  - Massive parallelism
- Compared to CPLDs:
  - Much higher logic density
  - SRAM-based configuration (loaded at startup)
  - Ideal for complex systems

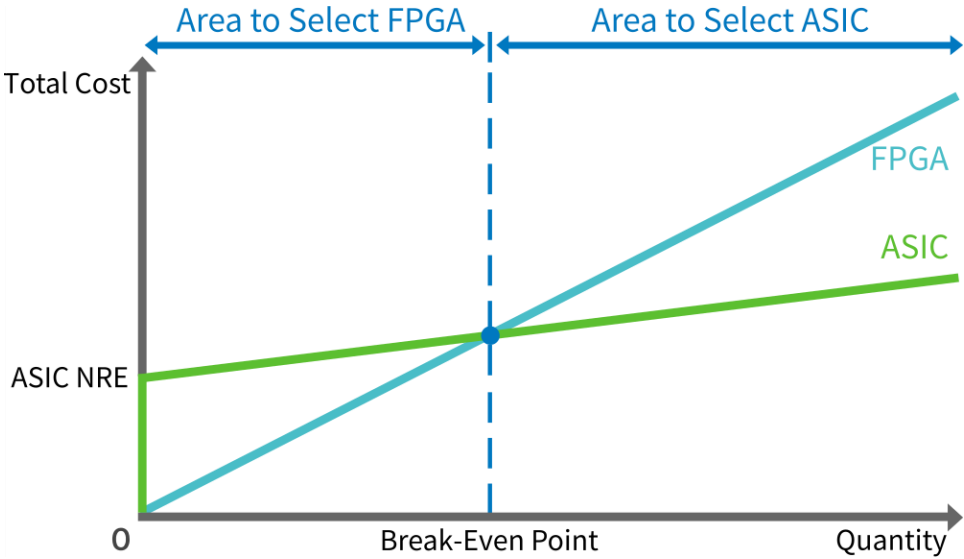




# Price point

|                                                                                   |                                                                   |                                                                                |                             |                                                    |                                     |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------|----------------------------------------------------|-------------------------------------|
|  | <b>Produttore Codice prodotto</b><br>Lattice<br>ICE40UL640-CM36AI | FPGA - Array di gate programmabile su campo FPGA ICE40-UltraLite 1.2V CBGA PKG | <a href="#">Scheda dati</a> | 243 A magazzino                                    | 1 2,58 €<br>25 2,24 €<br>100 2,12 € |
|  | <b>Produttore Codice prodotto</b><br>Lattice<br>ICE40UL1K-CM36AI  | FPGA - Array di gate programmabile su campo FPGA ICE40-UltraLite 1.2V CBGA PKG | <a href="#">Scheda dati</a> | 722 A magazzino                                    | 1 2,67 €<br>25 2,32 €<br>100 2,24 € |
|  | <b>Produttore Codice prodotto</b><br>Lattice<br>ICE40LP384-CM49   | FPGA - Array di gate programmabile su campo ICE40LP Ultra LowPwr 384 LUTs 1.2V | <a href="#">Scheda dati</a> | 836 A magazzino                                    | 1 2,67 €<br>25 2,32 €<br>100 2,24 € |
|  | <b>Produttore Codice prodotto</b><br>Lattice<br>ICE40LP384-CM36TR | FPGA - Array di gate programmabile su campo ICE40LP Ultra LowPwr 384 LUTs 1.2V | <a href="#">Scheda dati</a> | Tempo di consegna, se non a magazzino 24 settimane | Nastro continuo 20.000 2,68 €       |

|                                                                                    |                                                                         |                                                                |                             |                                                    |               |                                                             |                             |                 |                                                                              |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------|----------------------------------------------------|---------------|-------------------------------------------------------------|-----------------------------|-----------------|------------------------------------------------------------------------------|
|     | <b>Produttore Codice prodotto</b><br>Aglilex<br>AGIA035R39A2I3V         | FPGA - Array di gate programmabile su campo                    | <a href="#">Scheda dati</a> | Tempo di consegna diretta 16 settimane             | 3 38.140,14 € | Array di gate programmabile su campo FPGA ICE40-3 1.2V WLCS | <a href="#">Scheda dati</a> | 832 A magazzino | Nastro continuo 1 2,71 €<br>25 2,37 €<br>100 2,24 €<br>Nastrati 1.000 2,24 € |
|     | <b>Produttore Codice prodotto</b><br>Altera<br>1ST210E2F50I1VG          | FPGA - Array di gate programmabile su campo                    | <a href="#">Scheda dati</a> | Tempo di consegna diretta 16 settimane             | 3 37.496,86 € |                                                             |                             |                 |                                                                              |
|     | <b>Produttore Codice prodotto</b><br>Aglilex<br>AGIA035R39A2E2VB        | FPGA - Array di gate programmabile su campo                    | <a href="#">Scheda dati</a> | Tempo di consegna diretta 16 settimane             | 3 37.080,62 € |                                                             |                             |                 |                                                                              |
|    | <b>Produttore Codice prodotto</b><br>Aglilex<br>AGIA035R39A2E3E         | FPGA - Array di gate programmabile su campo                    | <a href="#">Scheda dati</a> | Tempo di consegna diretta 16 settimane             | 3 37.080,62 € |                                                             |                             |                 |                                                                              |
|  | <b>Produttore Codice prodotto</b><br>AMD / Xilinx<br>XCVU7P-L2FLVB2104E | FPGA - Array di gate programmabile su campo XCVU7P-L2FLVB2104E | <a href="#">Scheda dati</a> | Tempo di consegna, se non a magazzino 37 settimane | 1 38.475,97 € |                                                             |                             |                 |                                                                              |
|  | <b>Produttore Codice prodotto</b><br>AMD / Xilinx<br>XCVU7P-2FLVB2104I  | FPGA - Array di gate programmabile su campo XCVU7P-2FLVB2104I  | <a href="#">Scheda dati</a> | Tempo di consegna, se non a magazzino 37 settimane | 1 38.475,97 € |                                                             |                             |                 |                                                                              |

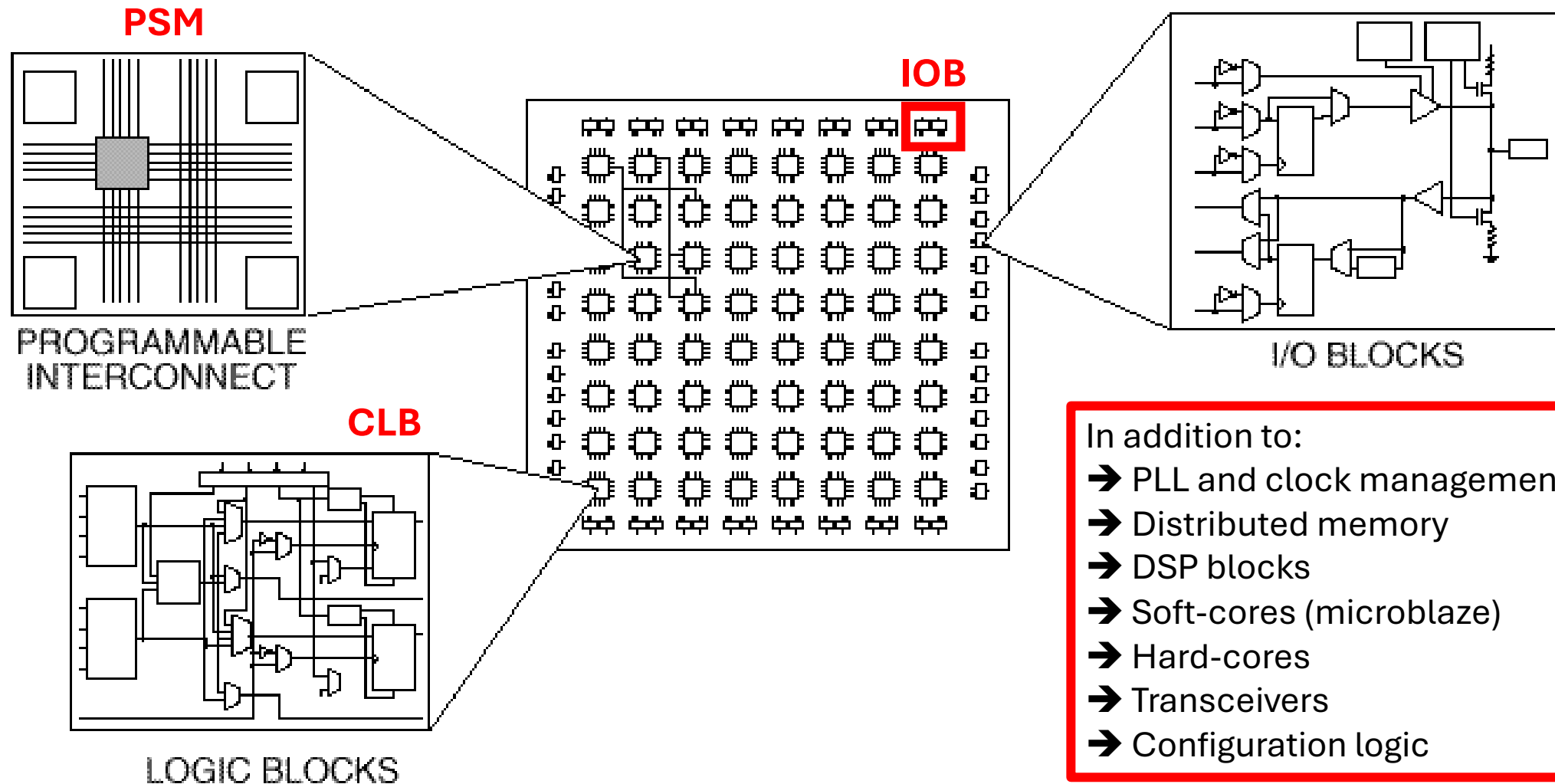


## Comparing FPGA and Dedicated ICs

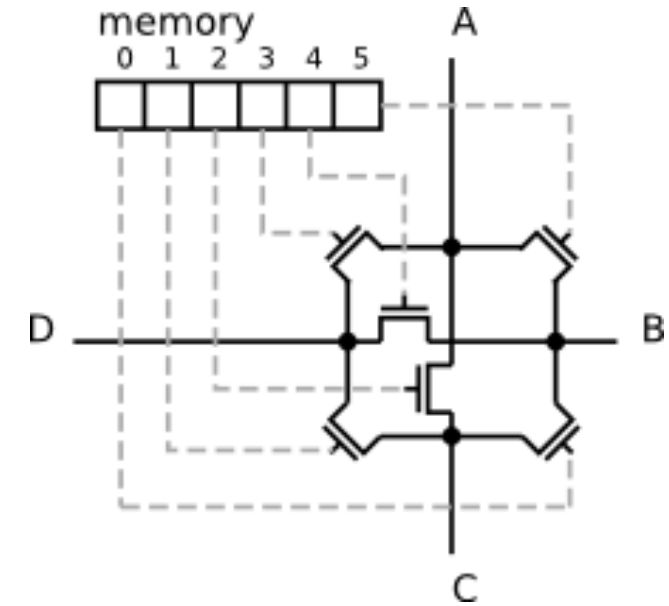
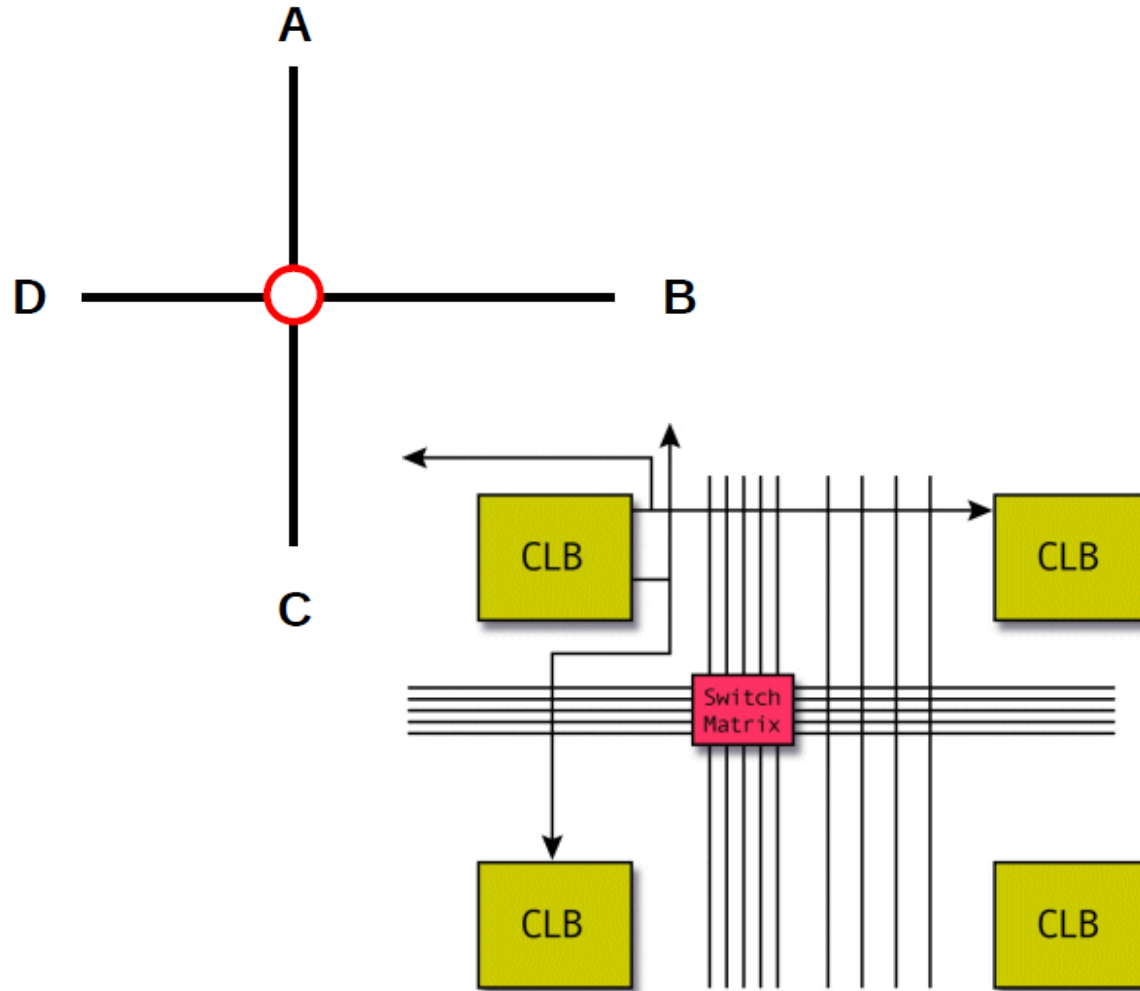
|                                  | FPGA            | Dedicated IC     |
|----------------------------------|-----------------|------------------|
| Up-front cost                    | Smaller         | Larger           |
| Per-unit cost                    | Larger          | Smaller          |
| Time to market                   | Shorter         | Longer           |
| Speed                            | Slower          | Faster           |
| Power consumption                | Greater         | Smaller          |
| Update in the field              | Straightforward | Very difficult   |
| Density                          | Lesser          | Greater          |
| Design Flow                      | Simpler         | More complex     |
| Granularity                      | Logic blocks    | Individual cells |
| Need for gate-level verification | Little          | Much             |
| Technology upgrade path          | Easier          | Harder           |
| Additional features              | Many            | Fewer            |



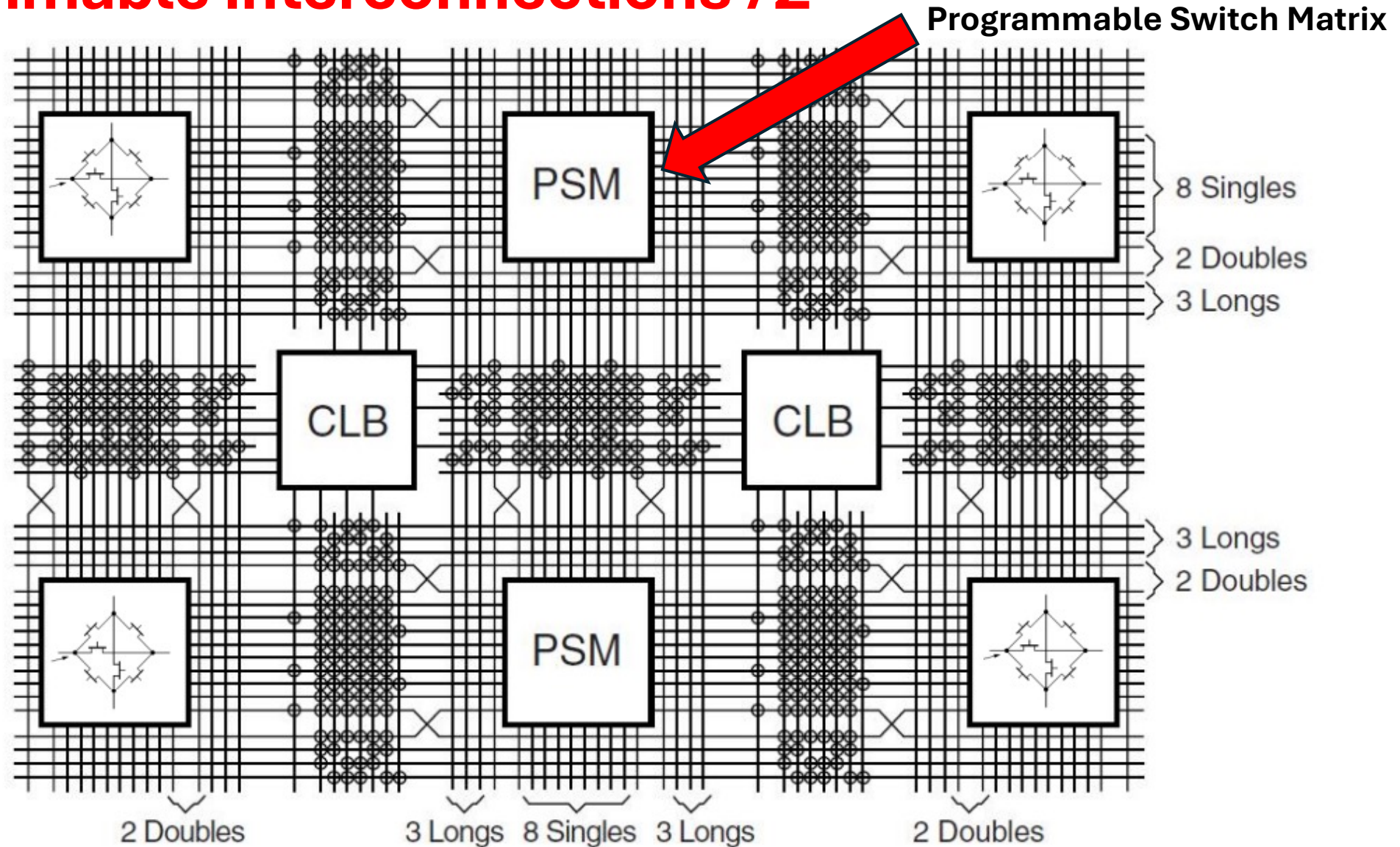
# Fundamental FPGA components



# Programmable interconnections /1



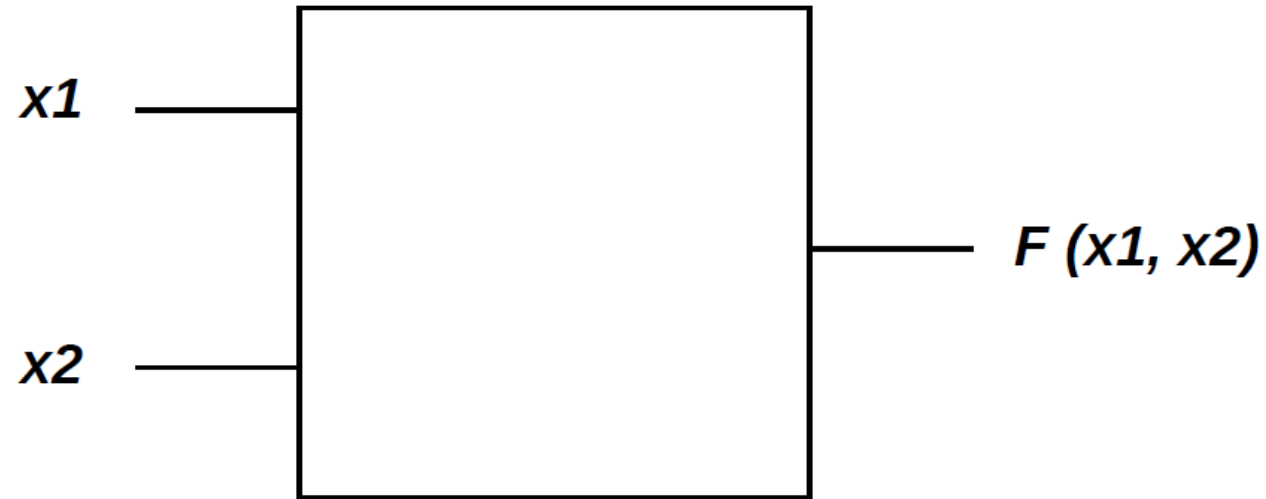
# Programmable interconnections /2





# Look-up tables (LUT) /1

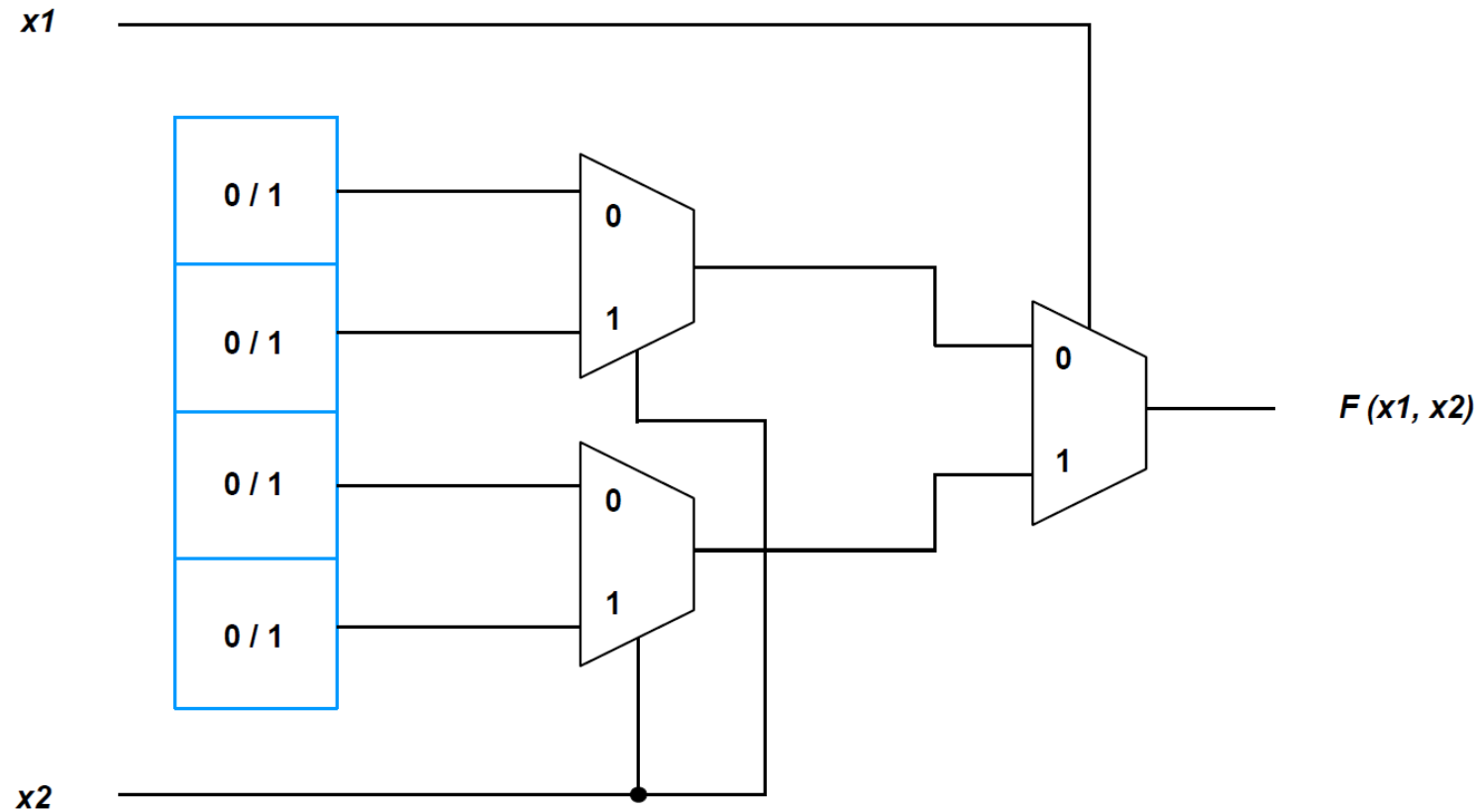
How to implement a 2-bit truth table (boolean logic) ?



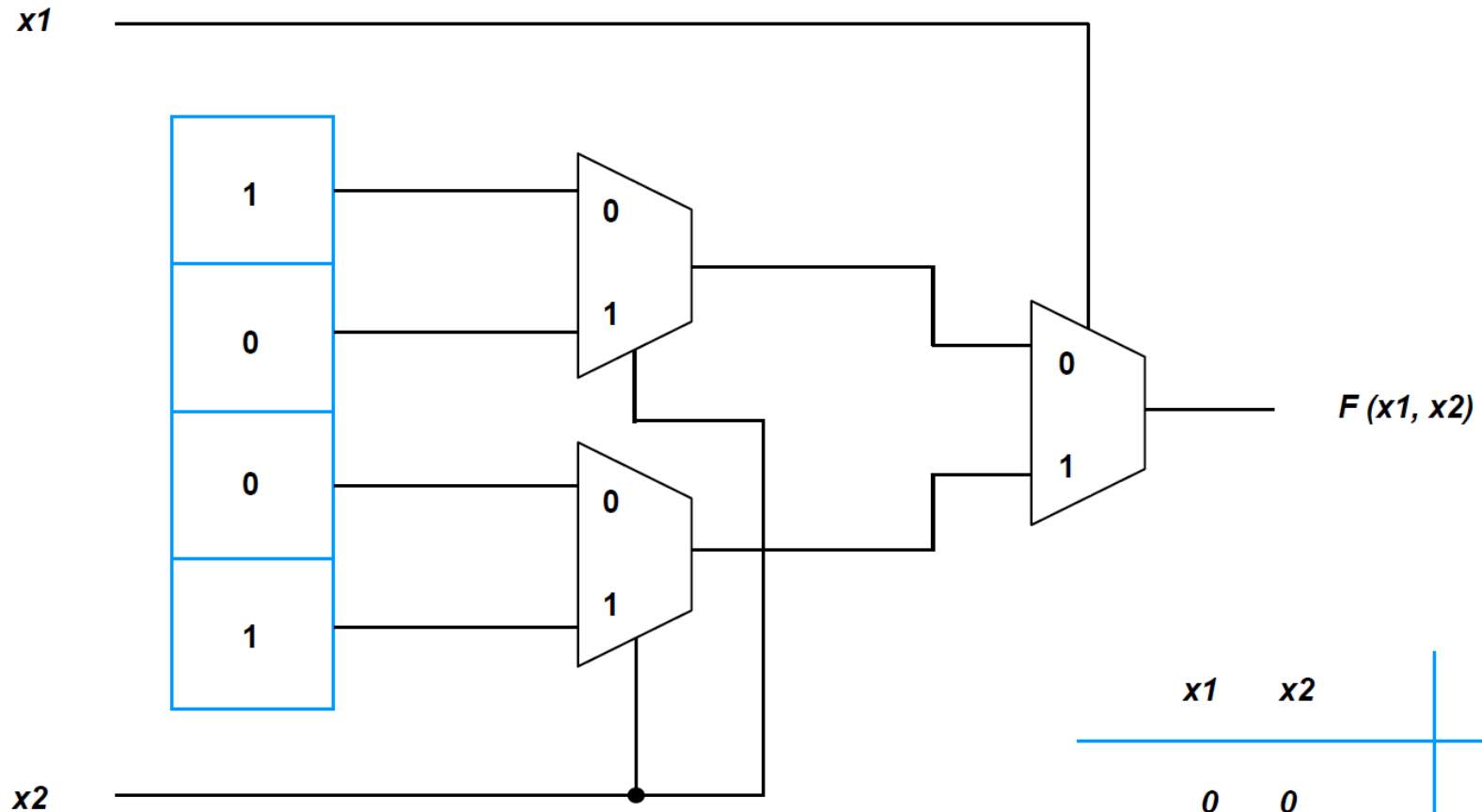
| XNOR |      |     |
|------|------|-----|
| $x1$ | $x2$ | $F$ |
| 0    | 0    | 1   |
| 0    | 1    | 0   |
| 1    | 0    | 0   |
| 1    | 1    | 1   |

# Look-up tables (LUT) /2

FPGA solution: write the truth table into a RAM !



# Look-up tables (LUT) /3



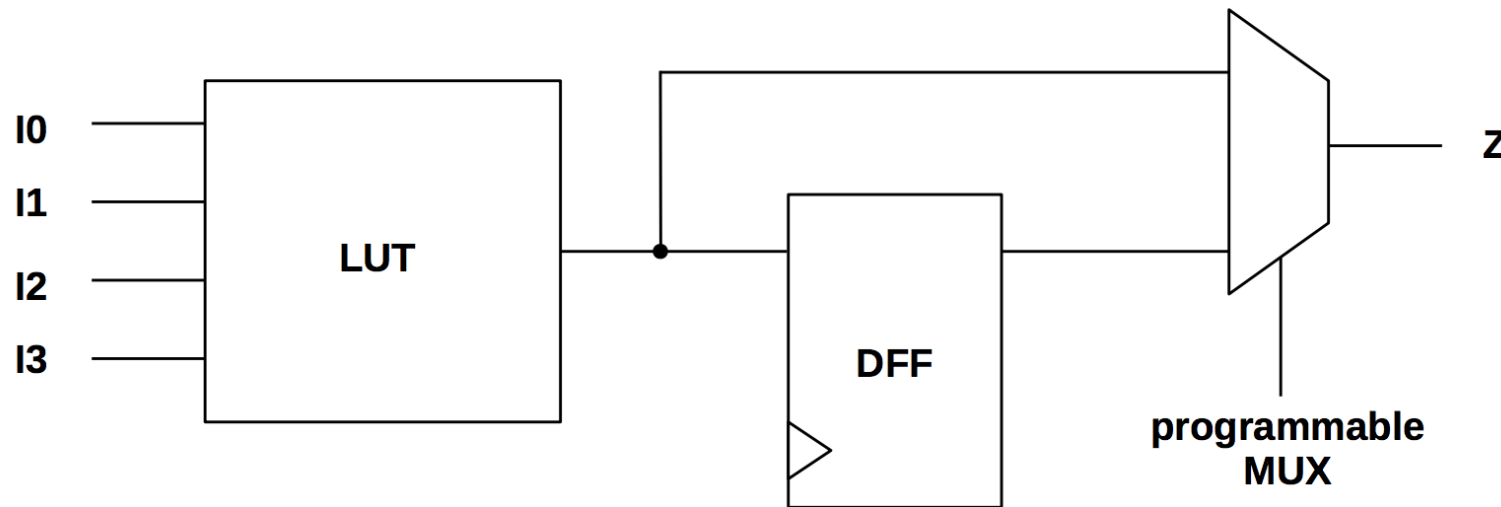
| $x_1$ | $x_2$ | $F$ |
|-------|-------|-----|
| 0     | 0     | 1   |
| 0     | 1     | 0   |
| 1     | 0     | 0   |
| 1     | 1     | 1   |

# Logic Cell (LC)

Xilinx terminology :

- **Logic Cell (LC)** = 1x LUT + 1 register (FF) + programmable MUX
- **LUT** implemented as 6-inputs, 64-bit RAM ← WHY?
- slice = 4x LUTs + 8 registers (FFs) + MUXs + fast-carry chain (CLA)
- **Congurable Logic Block (CLB)** = 2x slices
- Basic Element (BEL) = any device primitive

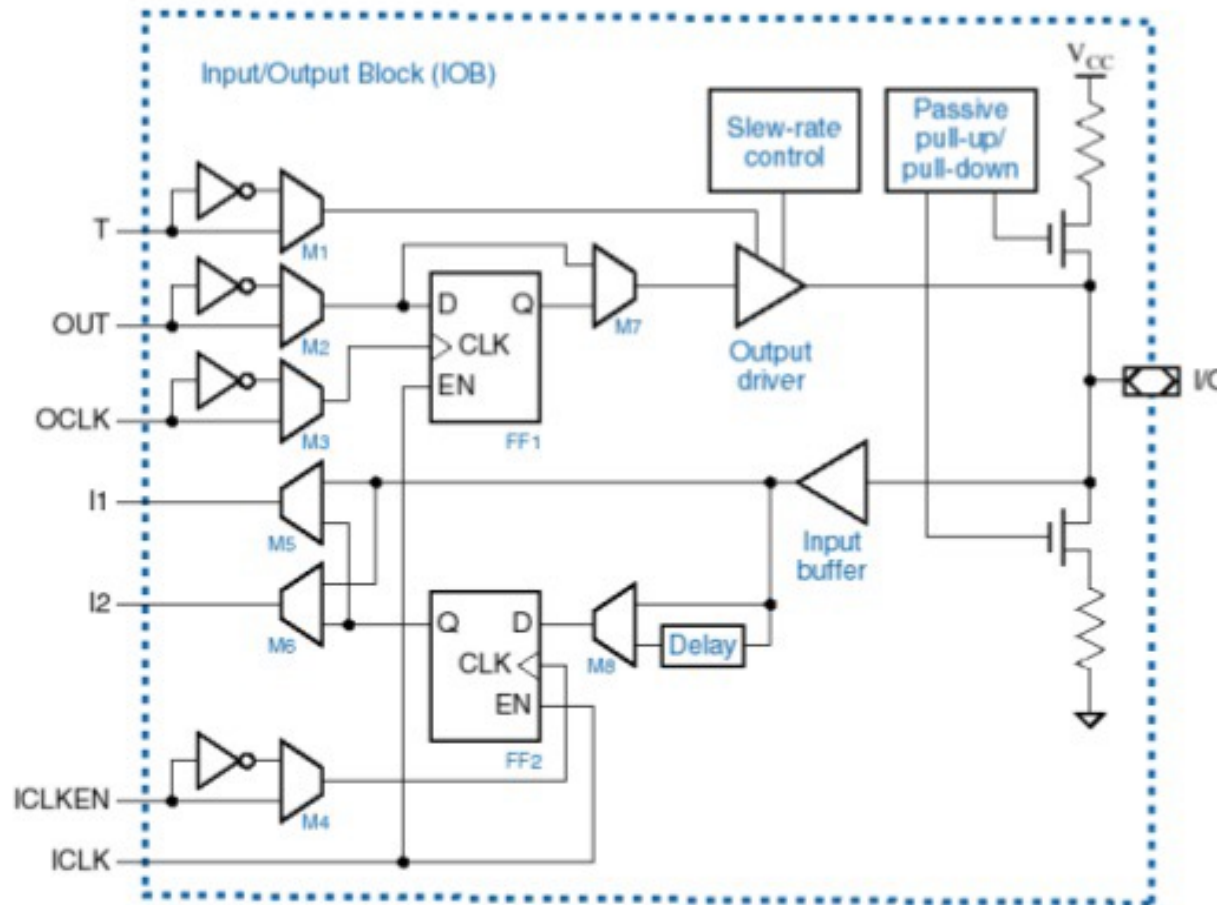
carry-lookahead adder





# Programmable I/O pads

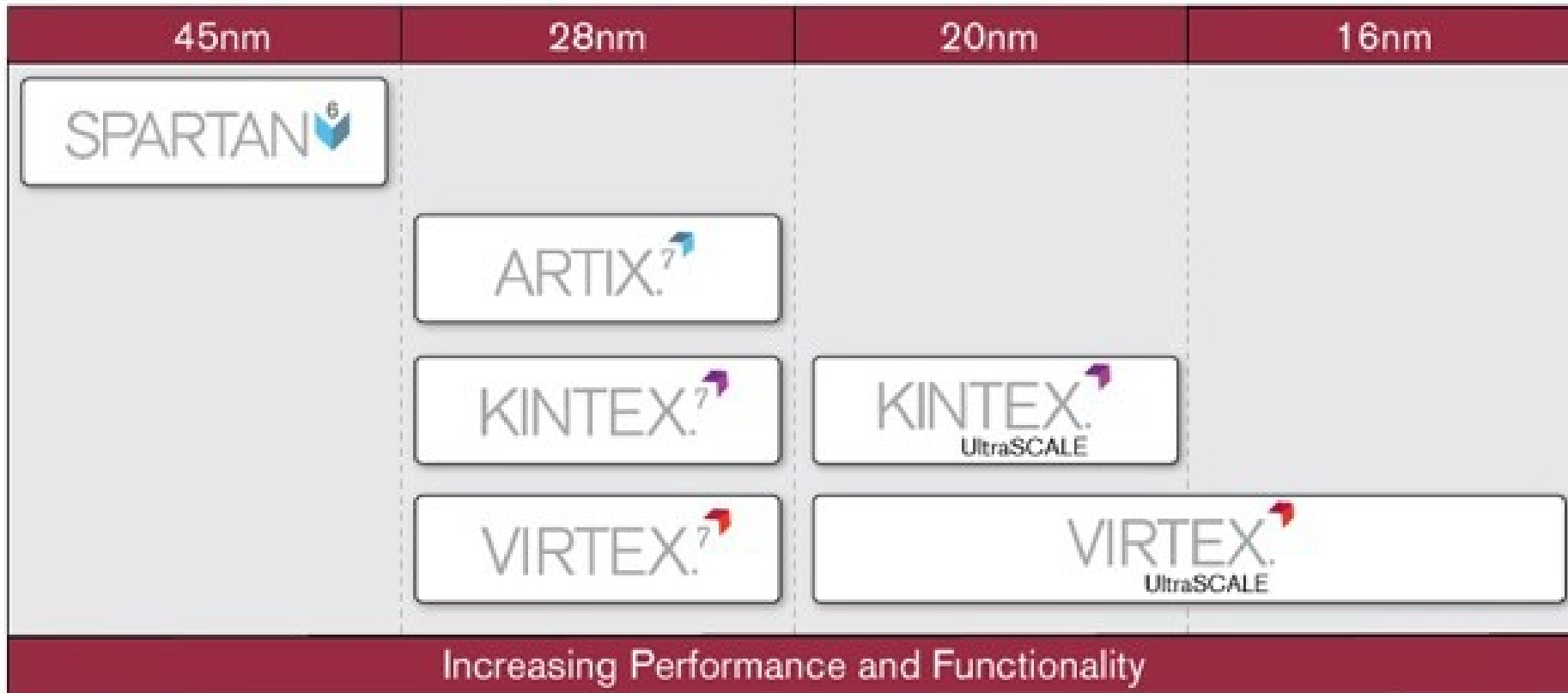
## I/O BLOCK



**Programmable  
pad**

44

# Xilinx FPGA line-up



# Xilinx 7-series family comparison

Different size, number of programmable logic cells, I/O capabilities, performance and cost depending on the chosen device family :

- Spartan 7 : lowest-cost, lowest-power, good I/O performance, lowest number of resources
- Artix 7 : low-power, medium number of resources and I/O, good choice for DSP
- Kintex 7 : best compromise between number of resources, I/O, speed, power and cost
- Virtex 7 : highest system performance (resources, speed, I/O) but also largest power consumption and highest cost

| Max. Capability   | Spartan-7           | Artix-7                                    | Kintex-7                                              | Virtex-7                         |
|-------------------|---------------------|--------------------------------------------|-------------------------------------------------------|----------------------------------|
| Logic Cells       | 102K                | 215K                                       | 478K                                                  | 1,955K                           |
| Block RAM         | 4.2 Mb              | 13 Mb                                      | 34 Mb                                                 | 68 Mb                            |
| DSP Slices        | 160                 | 740                                        | 1,920                                                 | 3,600                            |
| DSP Performance   | 176 GMAC/s          | 929 GMAC/s                                 | 2,845 GMAC/s                                          | 5,335 GMAC/s                     |
| MicroBlaze CPU    | 260 DMIPs           | 303 DMIPs                                  | 438 DMIPs                                             | 441 DMIPs                        |
| Transceivers      | –                   | 16                                         | 32                                                    | 96                               |
| Transceiver Speed | –                   | 6.6 Gb/s                                   | 12.5 Gb/s                                             | 28.05 Gb/s                       |
| Serial Bandwidth  | –                   | 211 Gb/s                                   | 800 Gb/s                                              | 2,784 Gb/s                       |
| PCIe Interface    | –                   | x4 Gen2                                    | x8 Gen2                                               | x8 Gen3                          |
| Memory Interface  | 800 Mb/s            | 1,066 Mb/s                                 | 1,866 Mb/s                                            | 1,866 Mb/s                       |
| I/O Pins          | 400                 | 500                                        | 500                                                   | 1,200                            |
| I/O Voltage       | 1.2V–3.3V           | 1.2V–3.3V                                  | 1.2V–3.3V                                             | 1.2V–3.3V                        |
| Package Options   | Low-Cost, Wire-Bond | Low-Cost, Wire-Bond,<br>Bare-Die Flip-Chip | Bare-Die Flip-Chip and High-<br>Performance Flip-Chip | Highest Performance<br>Flip-Chip |

# FPGA advantages and disadvantages

## Advantages

- Cheap
- Do anything!
- Super fast
- Field programmable
- Massively parallel
- High I/O count

## Disadvantages

- Expensive
- High power
- Volatile / BOOT time
- High pin count / BGA
- Complicated
- Many traps
- Complex tools
- Hard to choose / compare
- HDL “not easy” / intuitive

# How “programming” a FPGA?

|                                 |                                                                                                            |
|---------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Hardware Description HDL</b> | Describe what the hardware should do.                                                                      |
| <b>Syntax Check</b>             | Check Syntax                                                                                               |
| <b>Synthesis</b>                | “compiles” the design to transform HDL source into an architecture-specific design netlist. (connections)  |
| <b>Translate</b>                | Merges incoming netlists and constraints into a Xilinx design file<br>→ connections are merged with timing |
| <b>Map</b>                      | Fits the design into the available resources<br>→ tell the FPGA which gate structures should be used       |
| <b>Place &amp; Route</b>        | Places and routes the design to the timing constraints<br>→ decide, which gate is used for which function  |
| <b>Programming</b>              | Write configuration into the FPGA or configuration memory                                                  |



# Xilinx XDC – Xilinx Design Constraints file

XDC constraints are based on the standard **Synopsys Design Constraints (SDC)** format. SDC has been in use and evolving for more than 20 years, making it the most popular and proven format for describing design constraints.

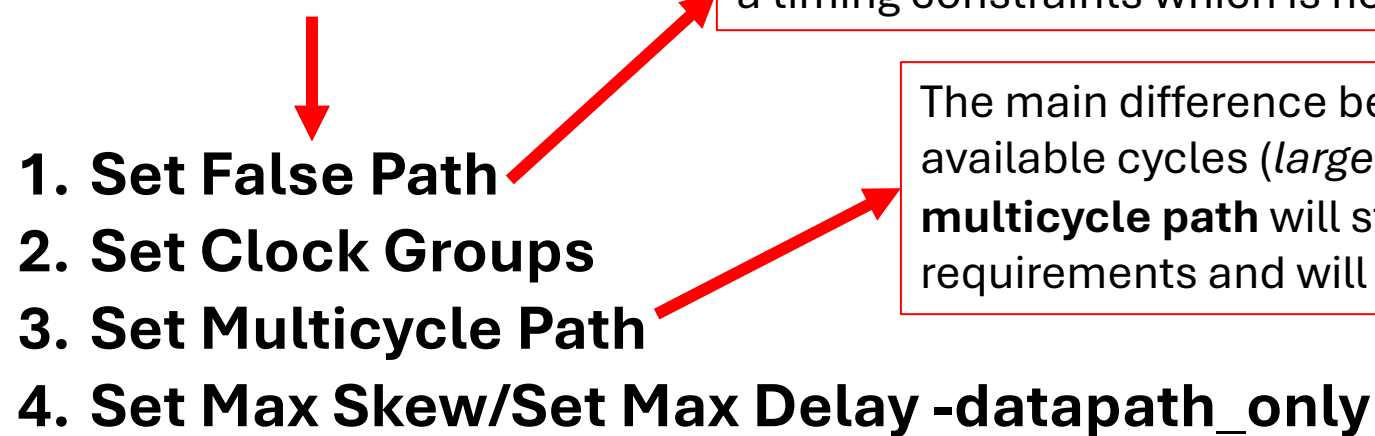
```
C:/Users/jcolvin/Documents/VivadoPrj/SwitchesAndLEDs/SwitchesAndLEDs.srcs/constrs_1/imports/VivadoXDCfiles/Arty_Master.xdc
1 ## This file is a general .xdc for the ARTY Rev. B
2 ## You can uncomment the lines by removing the two pound signs corresponding to used pins
3 ## You can rename the used ports (after "get_ports" in the curly braces) according to the names in your module
4 ## You can delete the pins you don't need for Project 2 since this XDC is a copy of the master one,
5 ## and not the original, or just leave them commented out
6 ## You might want to try out some of the other ports for yourself (depending on what board you have)
7 ## I uncommented and renamed the first switch and first LED for the beginning project
8
9 ##Switches
10 set_property -dict { PACKAGE_PIN A8      IOSTANDARD LVCMOS33 } [get_ports { sw }]; #IO_L12N_T1_MRCC_16 Sch=sw[0]
11 #set_property -dict { PACKAGE_PIN C11    IOSTANDARD LVCMOS33 } [get_ports { sw[1] }]; #IO_L13P_T2_MRCC_16 Sch=sw[1]
12 #set_property -dict { PACKAGE_PIN C10    IOSTANDARD LVCMOS33 } [get_ports { sw[2] }]; #IO_L13N_T2_MRCC_16 Sch=sw[2]
13 #set_property -dict { PACKAGE_PIN A10    IOSTANDARD LVCMOS33 } [get_ports { sw[3] }]; #IO_L14P_T2_SRCC_16 Sch=sw[3]
14
15 ##RGB LEDs
16 #set_property -dict { PACKAGE_PIN E1     IOSTANDARD LVCMOS33 } [get_ports { led0_b }]; #IO_L18N_T2_35 Sch=led0_b
17 #set_property -dict { PACKAGE_PIN F6     IOSTANDARD LVCMOS33 } [get_ports { led0_g }]; #IO_L19N_T3_VREF_35 Sch=led0_g
18 #set_property -dict { PACKAGE_PIN G6     IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO_L19P_T3_35 Sch=led0_r
19
20 ##LEDs
21 set_property -dict { PACKAGE_PIN H5      IOSTANDARD LVCMOS33 } [get_ports { led }]; #IO_L24N_T3_35 Sch=led[4]
22 #set_property -dict { PACKAGE_PIN J5     IOSTANDARD LVCMOS33 } [get_ports { led[1] }]; #IO_25_35 Sch=led[5]
23 #set_property -dict { PACKAGE_PIN T9     IOSTANDARD LVCMOS33 } [get_ports { led[2] }]; #IO_L24P_T3_A01_D17_14 Sch=led[6]
24 #set_property -dict { PACKAGE_PIN T10    IOSTANDARD LVCMOS33 } [get_ports { led[3] }]; #IO_L24N_T3_A00_D16_14 Sch=led[7]
25
26 ##Buttons

C:/Users/jcolvin/Documents/VivadoPrj/SwitchesAndLEDs/SwitchesAndLEDs.srcs/constrs_1/imports/VivadoXDCfiles/SwitchesAndLEDs.xdc
1 `timescale 1ns / 1ps
2 //Vivado automatically included our timescale
3 // Company: Digilent
4 // Engineer: James Colvin
5
6 // We can now edit our Vivado module. Here we can add our own constraints
7 //Vivado included most of this already for us
8 //What we need to add is the assign statement
9
10 module top(
11     input sw,
12     output led
13 );
14
15 assign led = sw; //this is the only line we need to add
16
17 endmodule
```

The names of the input and output in this module match the choose pins I want to use in the XDC file.

# XDC key concepts

1. Create clock
2. Define clocks interactions
3. Set input and output pins (and delays)
4. Set timing exceptions

- 
1. Set False Path
  2. Set Clock Groups
  3. Set Multicycle Path
  4. Set Max Skew/Set Max Delay -datapath\_only

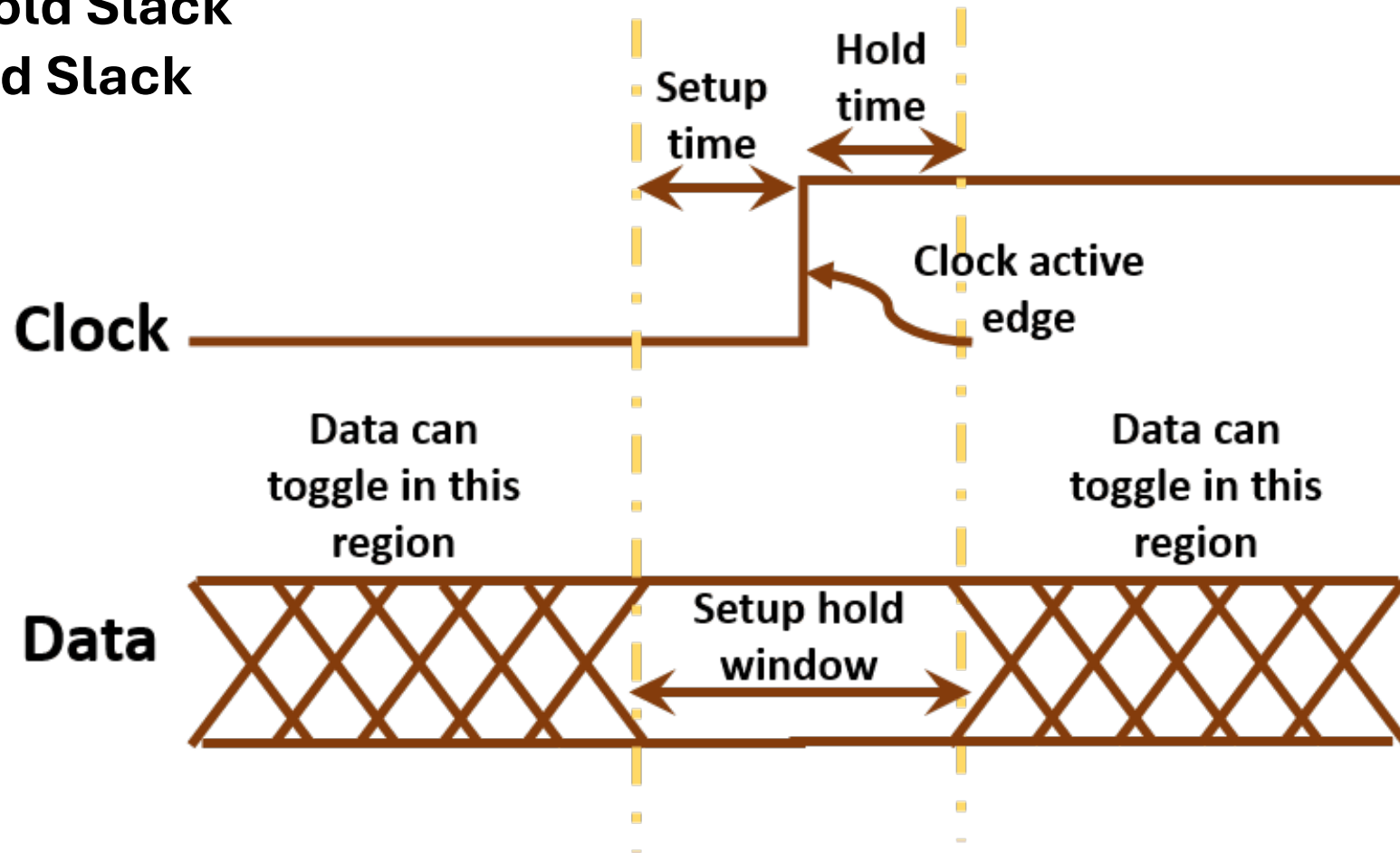
a timing constraints which is not required to be optimized for timing

The main difference between a **multicycle path** with many available cycles (*large n*) versus a **false path** is that the **multicycle path** will still be checked against setup and hold requirements and will still be included in the timing analysis.

```
set_multicycle_path <path_multiplier> [-setup|-hold] [-start|-end]  
[-from <startpoints>] [-to <endpoints>] [-through <pins|cells|nets>]
```

# WNS TNS WHS THS

1. Worst Negative Slack
2. Total Negative Slack
3. Works Hold Slack
4. Total Hold Slack



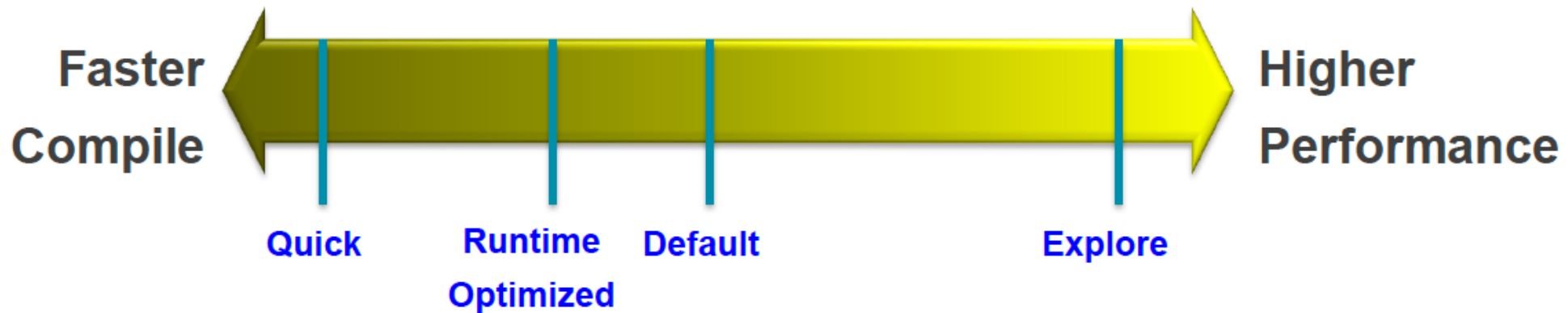
# Vivado Implementation Strategies and Directives

**Directive:** “directs” command behavior to try alternative algorithms

- Enables wider exploration of design solutions
- Applies to `opt_design`, `place_design`, `phys_opt_design`, `route_design`

**Strategy:** combination of implementation commands with directives

- Performance-centric: all commands use directives for higher performance
- Congestion-centric: all commands use directives that reduce congestion
- Flow-centric: modifies the implementation flow to add steps to Defaults
  - `power_opt_design`
  - post-route `phys_opt_design`



# Recommended Constraints Sequence

```
## Timing Assertions Section # Primary clocks  
# Virtual clocks  
# Generated clocks # Clock Groups  
# Bus Skew constraints  
# Input and output delay constraints
```

```
## Timing Exceptions Section # False Paths  
# Max Delay / Min Delay # Multicycle Paths  
# Case Analysis # Disable Timing
```

```
## Physical Constraints Section  
# located anywhere in the file, preferably before or after the timing  
constraints # or stored in a separate constraint file
```

# Example: Digilent Arty-A7 evaluation board

Device: Xilinx Artix-7

